



BAB II

TINJAUAN PUSTAKA

2.1 Arsitektur Perangkat Lunak

Sebagaimana dibahas dalam (Manchana, 2020; Barde, 2023), pemilihan arsitektur perangkat lunak berperan strategis dalam mendukung *scalability*, *maintainability*, serta fleksibilitas sistem di masa depan. Berbagai pendekatan telah dikembangkan, mulai dari *monolithic* yang sederhana hingga *microservices* yang lebih fleksibel, serta alternatif seperti *hybrid microservices* dan *modular monolith* untuk menjembatani kelemahan dari masing-masing arsitektur (Manchana, 2020; Goncalves, dkk., 2021; Godavarti, 2025). Selain itu, konsep *clean code* berperan penting dalam memastikan sistem tetap terstruktur, mudah dikelola, dan dapat berkembang sesuai kebutuhan.

2.1.1 *Monolithic*

Monolithic architecture adalah pendekatan tradisional dalam pengembangan perangkat lunak, di mana semua komponen sistem terintegrasi dalam satu unit besar (Barde, 2023). Keunggulannya terletak pada kesederhanaan pengembangan awal (Manchana, 2020), efisiensi dalam *debugging*, serta performa yang lebih baik karena tidak ada *overhead* komunikasi antar layanan (Thatikonda, dkk., 2024). Namun, seiring bertambahnya fitur dan skala sistem, *monolithic* menjadi sulit untuk diskalakan dan menyebabkan *bottleneck* dalam pengembangan, yang mendorong kebutuhan akan arsitektur yang lebih fleksibel seperti *microservices* (Blinowski, dkk., 2022).

Tantangan lain dari *monolithic* adalah keterikatan erat antar komponen, sehingga perubahan pada satu bagian sistem dapat memengaruhi keseluruhan aplikasi (Blinowski, dkk., 2022). Selain itu, kompleksitas kode cenderung akan meningkat seiring dengan bertambahnya fitur, sehingga menghambat *maintainability* dan pengelolaan versi yang lebih besar (Matlekovic dan Schneider-Kamp, 2022). Oleh karena itu, banyak organisasi mulai beralih ke pendekatan *microservices* untuk mengatasi masalah *scalability* dan meningkatkan *flexibility* dalam pengembangan perangkat lunak (Thatikonda, dkk., 2024).

2.1.2 *Microservices*

Microservices membagi sistem menjadi sejumlah layanan kecil yang berjalan secara independen dan berkomunikasi melalui API (*Application Programming Interface*), meningkatkan *scalability* serta memungkinkan pengembangan fitur secara terpisah. Setiap layanan dapat dikembangkan dengan teknologi yang berbeda, sehingga memungkinkan tim bekerja lebih cepat dan inovatif tanpa harus mempertahankan satu teknologi utama seperti pada *monolithic*. Keunggulan lain dari *microservices* adalah *fault tolerant*, di mana kegagalan satu layanan tidak memengaruhi keseluruhan sistem, sehingga meningkatkan ketahanan aplikasi (Blinowski, dkk., 2022; Thatikonda, dkk., 2024).

Meskipun menawarkan berbagai keunggulan, pendekatan *microservices* tidak lepas dari tantangan yang signifikan, khususnya dalam aspek komunikasi antar layanan. Peningkatan latensi akibat interaksi jaringan dapat menyebabkan *performance bottleneck*, sementara *observability* menjadi lebih sulit karena adanya banyak titik pemantauan dalam sistem yang terdistribusi (Manchana, 2020). Selain itu, pengelolaan *distributed database* memerlukan strategi yang matang untuk menjaga konsistensi data antar layanan (Blinowski, dkk., 2022). Oleh karena itu, pendekatan *hybrid* muncul sebagai solusi bagi organisasi yang ingin memanfaatkan fleksibilitas *microservices* tetapi tetap mempertahankan kestabilan sistem inti seperti pada *monolithic* (Godavarti, 2025).

2.1.3 Hybrid Microservices

Hybrid microservices merupakan pendekatan yang menggabungkan keunggulan *monolithic* dan *microservices*, memungkinkan organisasi untuk bertransisi secara bertahap tanpa mengorbankan stabilitas operasional. Pada pendekatan ini, bagian tertentu dari sistem tetap berada dalam *monolithic* sementara modul lain yang membutuhkan skalabilitas lebih tinggi dikembangkan sebagai *microservices* (Chitrakar, 2025). Dengan demikian, perusahaan dapat meningkatkan efisiensi dalam pengembangan sistem tanpa harus langsung bermigrasi sepenuhnya ke *microservices* yang lebih kompleks (Godavarti, 2025).

Meskipun menawarkan transisi yang lebih mulus, pendekatan *hybrid* tetap menghadapi tantangan utama sama seperti *microservices*, seperti kompleksitas komunikasi antar layanan, peningkatan *latency*, serta pengelolaan data yang tersebar di berbagai sistem. Oleh karena itu, meskipun *hybrid* memberikan keseimbangan yang baik bagi organisasi yang ingin melakukan migrasi bertahap, tetap diperlukan pertimbangan lebih lanjut sebelum benar-benar mengadopsinya (Thatikonda, dkk., 2024). Untuk mengatasi dilema ini, muncul konsep *Modular Monolith*, yang bertujuan memberikan fleksibilitas tanpa menghadapi *communication overhead* seperti pada *microservices* (Barde, 2023).

2.1.4 Modular Monolith

Modular Monolith hadir sebagai solusi bagi organisasi yang menghadapi dilema antara *monolithic* dan *microservices*, terutama dalam menentukan arsitektur awal yang fleksibel untuk jangka panjang. Pendekatan ini memungkinkan pemisahan layanan secara modular dalam satu *codebase*, sehingga fitur dapat dikembangkan tanpa ketergantungan yang kuat antar modul. Selain itu, juga dapat menghindari kompleksitas komunikasi seperti yang terjadi pada *microservices* (Goncalves, dkk., 2021; Barde, 2023; Pikkumäki, 2023; ?).

Keunggulan utama dari *Modular Monolith* adalah efisiensi dalam *maintainability* dan *debugging* karena semua komponen tetap berada dalam satu kesatuan yang mudah dikelola. Shopify dan Google adalah contoh perusahaan yang telah mengadopsi pendekatan ini untuk meningkatkan efisiensi pengembangan tanpa kehilangan kestabilan dalam sistem mereka. *Modular Monolith* memungkinkan sistem tetap modular sambil membuka peluang untuk transisi ke *microservices* di masa depan jika diperlukan (Barde, 2023; Tsehelidis, dkk., 2023; Skalicky, 2024; Su dan Li, 2024).

2.1.5 Hexagonal Architecture

Hexagonal Architecture adalah salah satu arsitektur pertama yang memecah *traditional layering*. *Hexagonal Architecture* atau yang biasa dikenal sebagai *adapter and port*, menggunakan konsep *port* sebagai *contract or interface* yang menentukan bagaimana aplikasi ini bekerja, dan konsep *adapter* sebagai implementasi dari *port*. Dengan dua konsep tersebut, *hexagonal architecture* menjaga *business logic* tetap *technology-agnostic* dan terjaga dari *dependency* dengan dunia luar. Selain *hexagonal architecture* ada beberapa arsitektur yang biasa digunakan untuk aplikasi besar seperti *Clean Architecture* (dikenalkan oleh Robert C. Martin pada 2017) atau *MVC Model-View-Controller* beserta turunannya yang direkomendasikan oleh Apple dan Google (Nunkesser, 2022).

Kurangnya standarisasi pada arsitektur *clean architecture* menjadikan terlalu banyak variasi dari *clean architecture* sehingga sulit untuk diimplementasikan secara global. Begitu pula dengan MVC dan turunannya yang sering kali implementasinya spesifik pada teknologi atau produk tertentu. Oleh karena itu, dalam penelitian ini mengusulkan penggunaan *hexagonal architecture* sebagai basis arsitektur dari *modular monolith* dibandingkan dengan *clean architecture* dan MVC yang biasanya digunakan sebagai *high-level design pattern* yang berfokus pada pemisahan UI (*User Interface*) (Nunkesser, 2022).

Pada *modular monolith*, pendekatan *Ports and Adapters* dalam *Hexagonal Architecture* memungkinkan sistem tetap modular tanpa meningkatkan ketergantungan antar modul, sehingga mempermudah integrasi dengan layanan eksternal. Keunggulan utama dari *Hexagonal Architecture* adalah fleksibilitas dalam mengelola *dependencies* serta memastikan keterpisahan antar lapisan aplikasi tanpa kehilangan efisiensi dalam pengembangan. Oleh karena itu, pendekatan ini menjadi solusi ideal bagi organisasi yang ingin menjaga modularitas sistem tanpa kompleksitas *microservices* penuh (Michail, 2023; Tsechelidis, dkk., 2023).

2.1.6 Clean Code

Clean Code berperan dalam menciptakan standarisasi kode, memastikan konsistensi dalam penulisan kode untuk meningkatkan *maintainability* dan keterbacaan sistem. Dengan prinsip seperti *Single Responsibility Principle* dan *Dependency Injection*, *clean code* membantu menjaga kualitas sistem agar tetap modular dan mudah diuji. Dalam konteks *modular monolith*, penerapan *clean code* memastikan bahwa setiap modul tetap terstruktur dengan baik, mengurangi *technical debt*, dan memudahkan transisi jika diperlukan di masa depan (Groeneveld, dkk., 2022; Ljung dan Gonzalez-Huerta, 2022). Selain itu, *clean code* juga digunakan untuk menghindari bias dalam penelitian ini.

Konsep-konsep yang ada pada *clean code* digunakan untuk menghindari bias dalam pengujian dan memastikan bahwa hasil pengujian itu berada pada level arsitektur bukan pada level penulisan kode. Prinsip-prinsip seperti DRY (*Don't Repeat Yourself*), *Keep It Simple*, *Meaningful Variable Names*, dan menghindari *comment* yang tidak perlu, diterapkan pada proses *refactoring* sejauh yang diterapkan oleh aplikasi *monolith*. Dengan demikian, pengujian aplikasi menjadi lebih objektif (Rahmesh, dkk., 2023).

2.2 *Code Quality Test*

Kualitas kode perangkat lunak memainkan peran penting dalam menjaga efisiensi dan keberlanjutan sistem selama siklus pengembangan. *Software quality* tidak hanya menentukan stabilitas dan skalabilitas aplikasi, tetapi juga memengaruhi *maintainability* serta *complexity* yang harus dikelola oleh tim pengembang. Oleh karena itu, berbagai teknik pengujian kode serta alat bantu seperti SonarQube, Postman, dan JMeter digunakan untuk memastikan bahwa kode memenuhi standar tertentu dan dapat berfungsi serta beradaptasi dengan perubahan kebutuhan sistem.

2.2.1 *Software Quality*

Maintainability

Maintainability mengacu pada sejauh mana kode perangkat lunak dapat diperbaiki, diperbarui, atau ditingkatkan tanpa menimbulkan masalah besar pada sistem yang ada (Sheta, 2023). Faktor-faktor yang memengaruhi *maintainability* meliputi struktur kode yang baik, dokumentasi yang jelas, serta pemisahan logika bisnis dan data agar kode lebih modular. Meskipun beberapa aspek *maintainability* tidak dapat diukur secara langsung dengan angka, *static code analysis* tetap menjadi pendekatan utama untuk mengidentifikasi potensi masalah dalam arsitektur dan mencegah kesalahan sejak awal pengembangan (Syed-Mohamad, dkk., 2025).

Complexity

Kompleksitas kode perangkat lunak berhubungan dengan tingkat kesulitan dalam memahami dan memelihara sistem, yang dapat berkontribusi terhadap meningkatnya kesalahan dan biaya pengembangan (Thatikonda, dkk., 2024). *Cyclomatic Complexity* digunakan untuk mengukur jumlah jalur independen dalam sebuah program, menunjukkan tingkat kerumitan dalam aliran eksekusi kode. Sementara itu, *Cognitive Complexity* seperti yang diterapkan oleh SonarQube berfokus pada aspek pemahaman manusia terhadap kode, dengan mempertimbangkan tingkat bersarangnya logika dan alur pemrograman yang kompleks (Campbell, 2023).

Performance

Kinerja perangkat lunak merupakan faktor kunci dalam menentukan efisiensi eksekusi kode dan kecepatan respons sistem terhadap pengguna. Performa aplikasi menjadi salah satu aspek yang menjadi bahan pertimbangan ketika memilih arsitektur. Semua arsitektur memiliki tujuan yang sama untuk meningkatkan performa aplikasi. Namun, tidak ada satu solusi untuk semua permasalahan, selalu ada *trade-off* yang menjadi risiko dan bahan pertimbangan (Godavarti, 2025). Seperti yang terjadi pada arsitektur *modular monolith* yang mengalami *performance degradation* dikarenakan komunikasi antar modul yang bertambah (Barde, 2023).

Scalability

Scalability mengacu pada kemampuan sistem untuk beradaptasi dengan peningkatan jumlah pengguna atau data tanpa mengalami penurunan performa yang signifikan (Thatikonda, dkk., 2024). *Monolithic architecture* memiliki keterbatasan dalam skalabilitas karena perubahan pada satu bagian sistem dapat berdampak pada keseluruhan aplikasi. *Modular Monolith* dan *Microservices* menawarkan alternatif dengan pendekatan yang lebih fleksibel dalam distribusi beban kerja (Barde, 2023).

2.2.2 *Testing*

API Testing

API Testing bertujuan untuk memastikan bahwa layanan API berjalan sesuai spesifikasi serta mampu berkomunikasi dengan baik antar sistem. Metode pengujian API dapat mencakup *automated testing* menggunakan alat seperti Postman. Selain memastikan fungsionalitas API, pengujian ini juga bertujuan untuk mengidentifikasi masalah seperti *error handling* yang buruk dan ketidakcocokan format data antar sistem yang berinteraksi (Kore, dkk., 2022).

Static Code Testing

Static Code Testing dilakukan tanpa menjalankan kode dan bertujuan untuk menemukan potensi *bug*, pelanggaran standar kode, serta analisis kompleksitas. SonarQube menjadi alat utama dalam pengujian ini dengan menganalisis *Cyclomatic Complexity* dan *Cognitive Complexity*, serta mengevaluasi *maintainability* berdasarkan pola kode yang tidak efisien atau rentan terhadap kesalahan (Campbell, 2023). Dengan pendekatan ini, pengembang dapat memperbaiki masalah sebelum kode di-deploy, sehingga meningkatkan kualitas perangkat lunak secara preventif.

Load Testing

Load testing adalah metode standar untuk menguji performa sistem dengan mensimulasikan pengguna dalam jumlah besar secara bersamaan. *Load testing* memungkinkan simulasi skenario pengguna yang realistis, membantu pengembang mengidentifikasi *bottleneck* yang muncul saat sistem mulai kewalahan dengan permintaan yang besar (Thatikonda, dkk., 2024). Alat seperti JMeter sangat efektif dalam skenario ini karena dapat melakukan simulasi ribuan permintaan dalam waktu bersamaan dan menghasilkan analisis menyeluruh mengenai *latency* serta *throughput* API (Khusaeri, dkk., 2023).

2.2.3 Software Quality Test Tools

Postman

Postman digunakan untuk melakukan pengujian API dengan cara otomatisasi permintaan dan validasi respons antar layanan. Alat ini mendukung *contract testing* untuk memastikan kompatibilitas API dalam sistem yang kompleks, serta memungkinkan pengujian dalam berbagai lingkungan konfigurasi. Dengan kemampuannya dalam mengelola skenario pengujian, Postman membantu tim pengembang dalam menjaga stabilitas sistem selama siklus pengembangan (Kore, dkk., 2022).

SonarQube

SonarQube merupakan alat analisis kode statis yang membantu mengidentifikasi *cyclomatic complexity*, *cognitive complexity*, dan *maintainability* dalam kode perangkat lunak (Campbell, 2023). Dengan memberikan metrik yang dapat dipahami oleh pengembang, SonarQube memungkinkan tindakan preventif dalam meningkatkan kualitas kode serta memastikan kepatuhan terhadap standar pengembangan. Alat ini juga dapat diintegrasikan ke dalam *pipeline* CI/CD untuk memberikan umpan balik langsung selama proses pengembangan perangkat lunak.

Salah satu metrik utama dalam SonarQube adalah *maintainability rating*, yang dihitung berdasarkan *technical debt ratio*. Nilai ini menunjukkan perbandingan antara beban teknis (*technical debt*) terhadap estimasi total biaya pengembangan aplikasi. Kategori penilaian tersebut ditampilkan dalam Tabel 2.1. Sementara untuk rumus yang digunakan menghitung *technical debt ratio* ditunjukkan pada Persamaan 2.1.

Tabel 2.1. *Maintainability Rating*

<i>Grade</i>	<i>Rate (technical debt ratio)</i>
A	$0\% \leq \text{TDR} \leq 5\%$
B	$5\% < \text{TDR} \leq 10\%$
C	$10\% < \text{TDR} \leq 20\%$
D	$20\% < \text{TDR} \leq 50\%$
E	$\text{TDR} > 50\%$

$$\text{technical debt ratio} = \frac{\text{technical debt}}{\text{development cost} \times \text{number of lines}} \quad (2.1)$$

Dalam persamaan tersebut:

- **technical debt** mengacu pada total waktu (dalam menit) yang dibutuhkan untuk memperbaiki semua *issue* yang terdeteksi,
- **development cost** adalah estimasi waktu (dalam menit) yang dibutuhkan untuk menulis satu baris kode,

- *number of lines* merupakan total baris kode yang dianalisis.

$$\text{cyclomatic complexity} = 1 + \text{number of conditional branch} \quad (2.2)$$

Kompleksitas logika dalam kode juga diukur melalui *cyclomatic complexity*, yang dihitung berdasarkan jumlah percabangan kondisional menggunakan Persamaan 2.2. Nilai total *cyclomatic complexity* dalam suatu aplikasi merupakan penjumlahan kompleksitas dari seluruh fungsi yang dianalisis. Sementara itu, *cognitive complexity* menilai seberapa sulit kode untuk dipahami oleh manusia. Berbeda dengan *cyclomatic complexity* yang berbasis jumlah percabangan, metrik ini mempertimbangkan struktur bersarang, alur kode yang tidak linear, serta penggunaan logika yang membingungkan seperti *goto*, *loop*, dan *branching* yang berlapis-lapis (Campbell, 2023).

JMeter

JMeter digunakan dalam *load testing*, khususnya untuk sistem berbasis API yang ingin mengukur *scalability* dan *performance* saat menghadapi lonjakan pengguna. JMeter dipilih karena popularitas dan performanya yang lebih baik dibandingkan dengan alat lain seperti *LoadRunner*, *Siege*, dan *Microsoft Visual Studio*. Alat ini mampu mensimulasikan ribuan permintaan secara bersamaan dan memberikan *insight* terkait *bottleneck* selama eksekusi layanan API (Khusaeri, dkk., 2023). Dengan fleksibilitasnya, JMeter membantu pengembang mengoptimalkan sistem sebelum digunakan dalam skenario produksi dengan beban tinggi.

Ada beberapa parameter yang dapat dikonfigurasi di JMeter, seperti jumlah pengguna (*threads*), *ramp-up*, dan *loop*. Jumlah pengguna menunjukkan berapa banyak pengguna yang akan mengakses sistem secara bersamaan. Periode *ramp-up* merupakan lama waktu yang dibutuhkan untuk keseluruhan *threads* dalam detik. *Loop* mengacu pada jumlah pengulangan yang dilakukan oleh masing-masing pengguna selama pengujian.

2.3 Server-side RESTful API

RESTful API (*Representational State Transfer API*) adalah arsitektur yang memungkinkan komunikasi antara *client* dan *server* dengan prinsip *statelessness*, *cacheability*, dan *uniform interface*. Dalam *REST API*, operasi utama dilakukan menggunakan metode *HTTP* seperti *GET*, *POST*, *PUT*, dan *DELETE*, yang memungkinkan interaksi yang efisien dengan data. Dibandingkan dengan *SOAP API*, *REST API* lebih ringan, cepat, dan fleksibel karena mendukung berbagai format seperti *JSON* dan *plain-text* (Kore, dkk., 2022). Pendekatan ini meningkatkan skalabilitas, mempermudah integrasi antar sistem, serta memungkinkan pengembang untuk membangun layanan yang dapat diakses dari berbagai platform, termasuk *web* dan *mobile*.

2.3.1 JavaScript (Node.js) *Environment*

JavaScript adalah bahasa pemrograman yang awalnya dikembangkan untuk kebutuhan *frontend*, namun dengan hadirnya Node.js, JavaScript kini banyak digunakan untuk *server-side programming* (Ardiansyah dan Fatwanto, 2022). Beberapa bahasa pemrograman lainnya juga mendukung pengembangan aplikasi *server-side* berbasis *REST API* seperti Java, Python, PHP, Go, dan beberapa bahasa lainnya. Namun setidaknya ada empat yang sering digunakan untuk aplikasi *microservice*, di antaranya yaitu Java, Node.js, Python, dan Go (Manchana, 2020).

Java lebih cocok untuk aplikasi *enterprise* dengan kompleksitas yang tinggi. Node.js dan Python memiliki dukungan dan lingkungan pengembangan yang memadai (Challapalli, dkk., 2021). Namun, dalam hal pengembangan *web*, JavaScript lebih banyak digunakan dibandingkan Python maupun Java (Chandra, 2025). Sementara itu, jika dibandingkan dengan bahasa yang lebih baru yaitu Go, dalam beberapa hal dan kondisi, perbedaan yang ditampilkan keduanya tidak terlalu signifikan (Ardiansyah dan Fatwanto, 2022).

2.3.2 NestJS (*Modularity*)

Menilik dari situs resmi NestJs, NestJs adalah *framework backend* berbasis Node.js yang dikembangkan dengan TypeScript dan dirancang untuk mendukung modularisasi aplikasi *backend*. Dengan menggunakan arsitektur berbasis *modular monolith*, memungkinkan pengembang untuk membangun layanan *backend* yang *scalable* dan *maintainable*, serta memisahkan logika bisnis ke dalam modul independen. Alternatif *framework* lain dalam ekosistem Node.js yang sering digunakan untuk pembuatan REST API adalah Express.js dan Hapi.js (Sutara dan Gunawan, 2024).

Express.js adalah *framework* minimalis yang banyak digunakan karena fleksibilitasnya, tetapi kurang mendukung arsitektur modular secara *native*, sehingga pengembang perlu mengelola struktur kode secara manual. Begitu pula dengan Hapi.js, walaupun keduanya menjadi favorit *framework* node.js dalam membangun REST API (Sutara dan Gunawan, 2024), kekurangannya dalam hal mendukung modularisasi menjadikannya perlu ekstra konfigurasi. Berbeda dengan Express.js dan Hapi.js, Nest.js memiliki pendekatan modularisasi yang kuat, penggunaan *Dependency Injection*, serta integrasi otomatis dengan berbagai *database* dan *testing framework*. Namun, salah satu keterbatasannya adalah kurva pembelajaran yang lebih tinggi, terutama bagi pengembang yang belum terbiasa dengan konsep TypeScript dan arsitektur modular.

2.3.3 JavaScript *Library*

Argon2

Argon2 adalah algoritma *hashing* yang dirancang untuk keamanan penyimpanan *password* dengan mempertimbangkan kecepatan pengisian memori dan efisiensi penggunaan unit pemrosesan. Algoritma ini dikembangkan untuk memberikan perlindungan terhadap *tradeoff attacks* dan serangan

berbasis *hardware* seperti *GPU cracking*. Bcrypt, PBKDF2, dan Scrypt adalah alternatif lain dari Argon2. Karena fleksibilitas dalam menentukan biaya komputasinya, Bcrypt lebih baik dari keduanya. Namun Argon2 lebih unggul dalam keamanan terhadap serangan berbasis *hardware*, serta memiliki fleksibilitas tinggi dalam konfigurasi parameter dibandingkan algoritma lain seperti Bcrypt dan PBKDF2 (George dan Mathew, 2021).

Prisma

Prisma adalah *Object-Relational Mapping (ORM)* modern yang dibuat khusus untuk Node.js, yang memungkinkan pengembang berinteraksi dengan *database* menggunakan kode pemrograman yang lebih bersih dan terstruktur. Penggunaan ORM itu penting karena membuat pengelolaan data lebih mudah, menghilangkan kesulitan menulis dan memahami *query SQL*. Dengan Prisma, pengembang dapat menghindari kesalahan dalam penulisan *query SQL* dan meningkatkan efisiensi pengelolaan data (Ramadhan dan Prayudi, 2025).

2.4 Studi Kasus (Tahsin App)

Tahsin App adalah aplikasi *e-learning* berbasis *RESTful API* yang dirancang untuk membantu pengguna dalam belajar membaca Al-Qur'an. Aplikasi ini menyediakan sistem manajemen pengguna, materi pembelajaran, dan evaluasi untuk mengoptimalkan pengalaman belajar tahsin. Dalam penelitian ini, Tahsin App dijadikan studi kasus untuk mengevaluasi bagaimana implementasi *backend* berbasis NestJS dan Node.js dapat mendukung fitur-fitur utama aplikasi dalam ekosistem JavaScript.

2.4.1 Arsitektur Aplikasi

Tahsin App menggunakan pendekatan *backend* berbasis *RESTful API*, yang memungkinkan komunikasi *stateless* antara klien dan server. Untuk pengelolaan data, aplikasi ini memanfaatkan Prisma ORM, sebuah ORM modern yang menyediakan integrasi efisien dengan berbagai sistem basis data. Selain itu, untuk keamanan pengguna, Tahsin App mengimplementasikan autentikasi berbasis tokenisasi, guna mengatur akses dan otorisasi dalam sistem.

2.4.2 Fitur Utama Tahsin App

Tahsin App memiliki berbagai fitur yang mendukung proses pembelajaran tahsin secara sistematis dan terstruktur. Manajemen pengguna memungkinkan registrasi, *login*, serta pengelolaan profil pengguna agar dapat mengikuti materi dengan personalisasi yang lebih baik. Materi pembelajaran dalam aplikasi ini mencakup berbagai tingkatan belajar membaca Al-Qur'an, yang bisa diakses sesuai dengan kebutuhan pengguna. Selain itu, sistem evaluasi kemajuan membantu pengguna mengetahui perkembangan mereka melalui penilaian yang dikumpulkan dari berbagai sesi latihan. Tahsin App juga memiliki fitur *logging*, yang berfungsi untuk mencatat aktivitas pengguna dan memberikan *insight* bagi pengelola aplikasi dalam meningkatkan performa sistem.

2.4.3 Relevansi Studi Kasus dengan Penelitian

Sebagai aplikasi *e-learning* yang berbasis *RESTful API*, Tahsin App menjadi contoh representatif dalam mengevaluasi arsitektur *backend* menggunakan NestJS dan Node.js. Studi kasus ini memungkinkan analisis lebih mendalam terhadap aspek performa dan skalabilitas, serta bagaimana arsitektur *backend* memengaruhi efisiensi sistem. Selain itu, penelitian ini juga memberikan kontribusi terhadap literatur arsitektur perangkat lunak, khususnya dalam pemanfaatan *RESTful API* berbasis JavaScript dalam aplikasi pendidikan.

2.5 Penelitian Terkait

Berbagai penelitian menunjukkan bahwa pendekatan modularisasi pada arsitektur perangkat lunak mampu menjembatani kesenjangan antara *monolithic* dan *microservices*. (Barde, 2023) menekankan pentingnya keputusan arsitektural terhadap kualitas sistem, namun tidak secara eksplisit membahas *modular monolith* di ekosistem JavaScript. (Su dan Li, 2024) mengidentifikasi *modular monolith* sebagai tren baru yang lebih ringan dari *microservices*, tetapi tidak menguji dampaknya secara kuantitatif. (Goncalves, dkk., 2021) mengevaluasi *refactoring monolith* menuju *microservices* dengan pendekatan modularisasi, namun fokusnya lebih pada transisi dan performa tanpa menyertakan metrik *maintainability* atau *complexity*. Sementara itu, (Godavarti, 2025) menawarkan pendekatan *hybrid* yang menggabungkan *monolith* dan *microservices*, tetapi tetap membawa *overhead* operasional khas *microservices*, berbeda dengan *modular monolith* yang lebih ringan. (Tsehelidis, dkk., 2023) menjadi satu-satunya studi yang menekankan pentingnya struktur modular dengan pendekatan *Hexagonal Architecture*, namun penelitian ini terbatas pada implementasi dan belum menyentuh pengukuran kuantitatif. Sebagian besar studi sebelumnya dilakukan menggunakan bahasa Java yang lazim digunakan dalam pengembangan sistem berskala besar atau kelas enterprise. Selain itu, fokus pengujian umumnya terbatas pada aspek performa, khususnya *response time*, tanpa melibatkan metrik kualitas perangkat lunak lainnya. Berbeda dari pendekatan tersebut, penelitian ini mengimplementasikan *modular monolith* dalam ekosistem JavaScript, yang lebih umum digunakan untuk aplikasi skala kecil. Tahsin App sebagai aplikasi *e-learning* berskala kecil dipilih sebagai studi kasus untuk mengevaluasi dampak modularisasi secara kuantitatif, meliputi metrik *maintainability*, *scalability*, *complexity*, dan *performance*. Dengan demikian, penelitian ini memperluas cakupan studi *modular monolith* di luar konteks *enterprise* dan ekosistem Java.